Лабораторная работа №10. Мониторинг, логирование и диагностика Docker

Тема: Состояние сервисов, журналы, статистика ресурсов и поиск неисправностей Docker Compose

Цель: Научиться диагностировать многоконтейнерное приложение Nginx → Express API → PostgreSQL и восстанавливать работу без потери данных.

Дисциплина: Docker и контейнеризация приложений

---

1. Общие сведения

Лабораторная работа №10. Мониторинг, логирование и диагностика Docker. Продолжительность: 4 академических часа (180 минут). Среда: Debian 13, Docker Engine, Docker Compose, Nginx, Express API и PostgreSQL 17.

Цель: научиться оценивать состояние сервисов Compose, просматривать логи и статистику ресурсов, выявлять причины неисправностей и восстанавливать работу приложения без удаления данных PostgreSQL.

2. Оборудование и предварительные условия

- Debian 13 с установленными Docker Engine и Compose plugin; доступ к терминалу.
- Рабочая копия Compose-проекта лабораторной №9 с nginx, api, db и REST-маршрутом /api/products.
- Доступный порт HTTP, настроенный в .env (в примерах — 8080).
- Выполняйте команды из каталога проекта. Если Docker требует административных прав, добавьте sudo.
- Выполняйте эксперименты только на учебном стенде. НЕ используйте docker compose down -v.

Не выводите переменные с паролями и не прикладывайте такие значения к отчёту. Для демонстрации достаточно проверить DB_HOST и DB_PORT.

3. Исходная архитектура

Клиент → http://127.0.0.1:8080 → nginx:80 → api:3000 → db:5432 (PostgreSQL). Сервисы связаны Docker DNS внутри одной Compose-сети. Данные БД находятся в именованном томе.

Объекты для диагностики

Объект	Что проверять
nginx	Опубликованный порт, HTTP, проксирование запросов
api	Процесс Node.js, логи, подключение к db
db	Состояние PostgreSQL, запуск и данные
volume	Сохранность товаров после перезапусков

4. Подготовка и создание независимой копии

Работу рекомендуется проводить на отдельной копии конфигурации, чтобы не менять файлы прошлой лабораторной. Не копируйте том PostgreSQL вручную: Docker управляет томами отдельно.

```
Подготовить каталог

cp -a ~/compose-course9 ~/compose-course10
cd ~/compose-course10
ls -la
```

Если рабочий каталог расположен иначе, замените путь. Убедитесь, что здесь есть compose.yaml, .env и каталоги api, nginx, db.

```
Проверить конфигурацию и запустить

docker compose config -q
docker compose up -d
docker compose ps -a
```

Ожидаемый результат: config -q завершается без ошибки; сервисы nginx, api, db запущены. Первый запуск PostgreSQL может занять некоторое время, дождитесь готовности БД.

```
Исходная функциональная проверка

curl -i http://127.0.0.1:8080/api/products
```

Если в вашем .env указан другой порт, подставьте его. Критерий: HTTP 200 и JSON-массив товаров. Если таблица ещё не готова, сначала проверьте логи db и api.

5. Задание 1. Состояния и метаданные контейнеров

```
Проверить работающие и остановленные контейнеры

docker ps -a
docker compose ps -a
```

Сравните общесистемный список и список текущего Compose-проекта. Зафиксируйте состояние каждого сервиса и его порты.

```
Посмотреть состояния через inspect

docker compose ps -q api
docker inspect --format "{{.State.Status}} / exit={{.State.ExitCode}}" $(docker compose
ps -q api)
```

Команда ps -q получает ID контейнера api; inspect показывает статус и код завершения. При нормальной работе ожидается running. Не считайте running доказательством исправности HTTP API.

6. Задание 2. Работа с логами

```
Логи сервисов

docker compose logs --tail 20 nginx
docker compose logs --tail 30 api
docker compose logs --tail 30 db
```

Найдите в логах сообщения о запуске сервисов. Удобно сначала искать в сервисе, который наблюдает симптом, затем переходить по цепочке зависимостей.

```
Следить за логами конкретного сервиса

docker compose logs -f --tail 10 nginx
```

Откройте второй терминал и выполните curl к API. Сравните время HTTP-запроса с записями в логах. Остановите режим слежения Ctrl+C — контейнер при этом не останавливается.

```
Логи с временными отметками

docker compose logs --since 5m --timestamps api
```

Параметр --since ограничивает период, --timestamps добавляет временную метку. Сохраните 2–3 строки, не раскрывающие секретные значения.

7. Задание 3. Мониторинг ресурсов

```
Посмотреть статистику

docker stats --no-stream
docker compose stats --no-stream
```

Найдите для nginx, api, db значения CPU %, MEM USAGE / LIMIT, NET I/O, BLOCK I/O, PIDS. Разовый снимок — наблюдение, а не полноценный нагрузочный тест.

Таблица для отчёта

Сервис	CPU %	MEM USAGE / LIMIT	PIDS
nginx	Заполнить	Заполнить	Заполнить
api	Заполнить	Заполнить	Заполнить
db	Заполнить	Заполнить	Заполнить

```
Просмотреть процессы Compose-проекта

docker compose top
```

Запишите, какие процессы работают в nginx, api и db. В минимальных контейнерных образах системные утилиты вроде ps могут отсутствовать; docker compose top выполняется на стороне Docker.

8. Задание 4. Проверка параметров и сети

```
Проверить конфигурацию и несекретные переменные

docker compose config -q
docker compose exec api printenv DB_HOST
docker compose exec api printenv DB_PORT
```

Ожидаемо: DB_HOST=db и DB_PORT=5432 (если проект №9 не был изменён). Если имя отличается, сравните его с именем сервиса базы данных в compose.yaml.

```
Проверка API и базы через внешнее HTTP API

curl -i http://127.0.0.1:8080/api/products
docker compose ps -a
```

Внешняя проверка API подтверждает прохождение запроса через Nginx и работу SQL. При наличии ошибки дополнительно анализируйте сообщения api и db.

9. Диагностический эксперимент №1: остановка API

Эксперимент обратим и не затрагивает том БД. Остановите только сервис api; сначала зафиксируйте нормальный ответ HTTP.

```
Намеренно остановить API

docker compose stop api
docker compose ps -a
curl -i http://127.0.0.1:8080/api/products
```

Ожидается недоступность маршрута через Nginx, обычно HTTP 502 Bad Gateway. Возможный статус зависит от версии и конфигурации Nginx. В отчёте укажите фактически увиденный статус.

```
Найти симптом в логах

docker compose logs --tail 20 nginx
docker compose logs --tail 20 api
```

Сопоставьте HTTP-симптом с остановленным сервисом api. Запишите доказательства: статус сервиса, HTTP-ответ и сообщение Nginx.

```
Восстановить API

docker compose start api
docker compose ps
curl -i http://127.0.0.1:8080/api/products
```

Критерий восстановления: api снова работает, HTTP-запрос возвращает 200 и JSON. Если проблема сохраняется, просмотрите api и db logs.

10. Диагностический эксперимент №2: неправильный DB_HOST

Сделайте резервную копию конфигурации, измените только значение DB_HOST на заведомо неверное имя и примените конфигурацию пересозданием API; не меняйте переменные PostgreSQL.

```
Сохранить копию конфигурации

cp compose.yaml compose.yaml.bak
cp .env .env.bak
cp api.env api.env.bak 2>/dev/null || true
```

Параметр DB_HOST может быть задан в .env, api.env или непосредственно в compose.yaml. Определите актуальное место по файлам проекта. Если api.env отсутствует, сообщение cp можно игнорировать.

```
Временно назначить неверный DB_HOST

grep -n "DB_HOST" .env api.env compose.yaml 2>/dev/null
```

В найденном файле измените значение DB_HOST на absent-db. Не заменяйте переменную во всех файлах подряд: исправляйте реально используемый источник. Затем проверьте итоговую конфигурацию без печати секретов.

```
Применить изменение сервиса API

docker compose config -q
docker compose up -d --force-recreate api
docker compose exec api printenv DB_HOST
curl -i http://127.0.0.1:8080/api/products
```

Ожидается DB_HOST=absent-db и ошибка запроса к базе (код HTTP зависит от реализации обработчика ошибок). Если API вовсе не запустился, проверьте ps -a и логи api. Не утверждайте, что ошибка всегда равна 500.

```
Найти причину

docker compose ps -a
docker compose logs --tail 40 api
docker compose logs --tail 20 db
```

Ищите сообщение разрешения DNS-имени (например, ENOTFOUND) или подключения. Запишите ошибочное значение DB_HOST и объясните, почему localhost не заменяет сервис db.

```
Восстановить файлы и сервис

cp compose.yaml.bak compose.yaml
cp .env.bak .env
if [ -f api.env.bak ]; then cp api.env.bak api.env; fi
docker compose config -q
docker compose up -d --force-recreate api
```

```
curl -i http://127.0.0.1:8080/api/products
```

Убедитесь, что значение DB_HOST вернулось к db и HTTP снова работает. Если API стартовал раньше готовности PostgreSQL, подождите и повторите запрос. Резервные копии можно удалить после проверки, если они не содержат учебных секретов.

11. Индивидуальное задание

- Составьте краткий алгоритм диагностики ошибки «Nginx отвечает, но список товаров не загружается».
- Продемонстрируйте вывод docker compose ps, двух команд logs и одного снимка stats.
- Назовите одну причину сбоя и покажите, каким фактом она подтверждается.
- Предложите безопасный способ восстановления без удаления Docker Volume.

12. Итоговая проверка

Финальная проверка проекта

```
docker compose config -q
docker compose up -d
docker compose ps -a
curl -i http://127.0.0.1:8080/api/products
docker compose stats --no-stream
```

Все три сервиса должны быть в нормальном рабочем состоянии, API отвечает HTTP 200 с JSON-массивом. Сохранность пользовательских данных проверяется повторным чтением ранее созданных товаров. В этой работе не выполнять docker compose down -v.

13. Требования к отчёту

- Тема, цель, версия Docker Engine/Compose и схема nginx → api → db.
- Скриншоты ps -a и итогового запроса curl.
- Краткая таблица ресурсов из docker stats.
- Логи nginx и api для двух диагностических экспериментов (без секретов).
- Таблица «симптом → проверка → первопричина → восстановление».
- Итоговый вывод: какие инструменты лучше подходят для состояния, логов и метрик.

14. Контрольные вопросы

1. Чем docker ps отличается от docker compose ps?
2. Почему running не гарантирует работоспособность API?
3. Для чего нужны docker compose logs --tail и --since?
4. Что показывает docker stats?
5. Почему DB_HOST=localhost не подходит для связи отдельных контейнеров?
6. Какая последовательность проверки при HTTP 502?
7. Что делает docker compose up -d --force-recreate api?
8. Почему опасно удалять томы при диагностике?

15. Критерии оценивания

Критерии

Критерий	Баллы

Критерий	Баллы
Проверка начального состояния и конфигурации	15
Чтение и анализ логов	20
Мониторинг ресурсов	15
Эксперимент: остановка и восстановление API	20
Эксперимент: неправильный DB_HOST и восстановление	20
Отчёт и контрольные вопросы	10
Итого	100

Официальная справка: https://docs.docker.com/reference/cli/docker/container/logs/ ; https://docs.docker.com/reference/cli/docker/container/stats/ ; https://docs.docker.com/reference/cli/docker/compose/logs/ .